# Worksheet 01 on Regression Part 1

## ➢ Objectives

To understand and implement different types of regression models.

## ➢ Machine Learning

In **AI** we are **mimicking human intellect** and **behavior**

In **ML** we are making machines to **learning without explicitly being programmed**

In **DL** we are making machines to **learning** based on **artificial neural networks**

## ➢ Supervised Machine Learning

A type of machine learning where the model is trained on labeled data.

The algorithm learns to map input features to the correct output based on provided examples.

**Key Characteristics:**

- Requires labeled datasets (input-output pairs).
- Used for **predictive modeling**.
- Common algorithms: Linear Regression, Logistic Regression, Decision Trees, Support Vector Machines (SVM).

**Applications:**

- Spam detection (classification).
- House price prediction (regression).
- Image recognition.

- ## Unsupervised Machine Learning

A type of machine learning where the model is trained on unlabeled data.

The algorithm identifies patterns, structures, or relationships in the data without explicit guidance.

**Key Characteristics:**

- No labeled data required.
- Used for exploratory analysis and pattern discovery.
- Common algorithms: K-Means Clustering, Hierarchical Clustering, Principal Component Analysis (PCA).

**Applications:**

- Customer segmentation (clustering).

- Dimensionality reduction for data visualization.

## ➢ Regression

A supervised learning technique used to predict continuous numerical values based on input features.

Models the relationship between dependent (target) and independent (input) variables.

**Types of Regression:**

- Linear Regression
  - Simple Linear Regression
  - Multiple Linear Regression
- Polynomial Regression

**Key applications:**

- Predicting house prices
- Forecasting stock prices
- Estimating sales revenue

**Dependent variable**: Whose values depend on the other variables

**Independent variable**: Who forms the basis of estimation

## ➢ Evaluation metrics for Regression models

### Why Evaluate Regression Models?

- To measure how well the model predicts the target variable.
- To compare different models and select the best one.
- To ensure the model generalizes well to unseen data.

### Common Regression Metrics

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R-Squared ($R^2$)

## ➢ Mean Squared Error (MSE )

- Measures the average squared difference between predicted and actual values.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

## ➢ Root Mean Squared Error (RMSE)

- Square root of MSE
- Lower MSE/RMSE indicates better performance.
- RMSE is in the same units as the target variable, making it more interpretable than MSE.

$$\text{RMSE} = \sqrt{\text{MSE}}$$

## ➢ Mean Absolute Error (MAE)

- MAE measures the average absolute difference between predicted and actual values.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

  - $y_i$: Actual value.
  - $\hat{y}_i$: Predicted value.
  - n: Number of observations
- Lower MAE indicates better model performance.
- Easy to interpret as it is in the same units as the target variable
- When to use MAE:
  - Use MAE when your dataset contains outliers and you want a metric that is less sensitive to them.
  - MAE is particularly useful in applications where all errors (large or small) should be treated equally.

## ➢ $R^2$ score

- Another useful metric that you will see in software output is the coefficient of determination, also called the R-squared statistic or $R^2$. R-squared ranges from 0 to 1.
- The $R^2$ (R-squared) metric measures how well a regression model explains the variance in the dependent variable. It provides insight into the goodness of fit.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} \qquad SS_{res} = \text{Sum of Squared Residuals} = \sum(y_i - \hat{y}_i)^2 \qquad SS_{tot} = \text{Total Sum of Squares} = \sum(y_i - \bar{y})^2 \qquad R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

- It measures the proportion of variation in the data that is accounted for in the model.
- Interpretation of $R^2$:
  - $R^2 = 1 \rightarrow$ Perfect fit (all variance explained).
  - $R^2 = 0 \rightarrow$ Model does not explain variance beyond the mean.
  - Negative $R^2 \rightarrow$ Model performs worse than simply using the mean.

## ➢ Simple Linear Regression

**Simple linear regression** provides a model of the **relationship between** the **magnitude of one variable** and **that of a second**—for example, as X increases, Y also increases. Or as X increases, Y decreases.

**Correlation** is another way to measure how two variables are related.

The difference between regression and correlation is:

- Correlation measures the **strength of an association** between two variables.
- Regression **quantifies the nature of the relationship**.

Simple linear regression estimates how much Y will change when X changes by a certain amount. With the correlation coefficient, the variables X and Y are inter-changeable.

With regression, we are trying to predict the Y variable from X using a linear relationship (i.e., a line):

$$y=b0+b1X$$

The symbol b0 is known as the **intercept** (or constant), and the symbol b1 as the **slope for X or regression coefficient**.

- **b0** is the value of dependent when independent variable value is zero
- **b1** tells the rate change in dependent variable with unit change in independent variable

The Y variable is known as the **response or dependent variable** since it depends on X.

The X variable is known as the **predictor or independent** variable.

The **machine learning community** tends to use other terms, calling Y the **target** and X a **feature vector**.

### Simple Linear Regression

```python
# Required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

```python
# Step 1: Create sample dataset (Hours studied vs Exam Score)
data = {
    'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12],
    'Exam_Score':    [50, 55, 62, 68, 75, 80, 84, 88, 91, 94, 96, 98]
}
```

```python
df = pd.DataFrame(data)
print("Dataset (Hours Studied vs Exam Score):")
print(df)
print()
```

```
Dataset (Hours Studied vs Exam Score):
    Hours_Studied  Exam_Score
0               1          50
1               2          55
2               3          62
3               4          68
4               5          75
5               6          80
6               7          84
7               8          88
8               9          91
9              10          94
10             11          96
11             12          98
```

```python
# Step 2: Prepare features (X) and target (y)
X = df[['Hours_Studied']]   # 2D structure (DataFrame) - required by sklearn
y = df['Exam_Score']        # 1D structure (Series)
```

```python
# Step 3: Split data into training and testing sets
# 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,          # 20% for testing
    random_state=42         # same split every time
)

print(f"Training samples: {len(X_train)}")
print(f"Testing samples:  {len(X_test)}")
print()
```

```
Training samples: 9
Testing samples:  3
```

```python
# Step 4: Create and train the model (only on training data!)
model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

```python
# Step 5: Show the learned regression equation
b0 = model.intercept_   # y-intercept
b1 = model.coef_[0]     # slope

print("Learned Regression Line:")
print(f"Exam_Score = {b0:.2f} + {b1:.2f} × Hours_Studied")
print()
```

```
Learned Regression Line:
Exam_Score = 50.65 + 4.38 × Hours_Studied
```

```python
# Step 6: Make predictions on test data (unseen data)
y_test_pred = model.predict(X_test)

# Calculate metrics on the TEST set
mae  = mean_absolute_error(y_test, y_test_pred)
mse  = mean_squared_error(y_test, y_test_pred)
rmse = np.sqrt(mse)
r2   = r2_score(y_test, y_test_pred)

print("┌──────────────────────────────────────────────────────────┐")
print("│ PERFORMANCE METRICS - SIMPLE LINEAR REGRESSION           │")
print("├──────────────────────────────────────────────────────────┤")
print(f"│ MAE  (Mean Absolute Error)     : {mae:>10.2f}              │")
print(f"│ MSE  (Mean Squared Error)      : {mse:>10.2f}              │")
print(f"│ RMSE (Root Mean Squared Error) : {rmse:>10.2f}             │")
print(f"│ R² Score                       : {r2:>10.4f} ({r2*100:>5.1f}%)     │")
print("└──────────────────────────────────────────────────────────┘")
print()
```

```
┌──────────────────────────────────────────────────────────┐
│ PERFORMANCE METRICS - SIMPLE LINEAR REGRESSION           │
├──────────────────────────────────────────────────────────┤
│ MAE  (Mean Absolute Error)     :       2.75              │
│ MSE  (Mean Squared Error)      :      11.11              │
│ RMSE (Root Mean Squared Error) :       3.33              │
│ R² Score                       :     0.9753 ( 97.5%)     │
```

```python
# Step 6: Make predictions on test data (unseen data)
y_test_pred = model.predict(X_test)

# Show actual vs predicted for test students
print("Test Set - Actual vs Predicted:")
test_results = pd.DataFrame({
    'Hours': X_test['Hours_Studied'],
    'Actual Score': y_test,
    'Predicted Score': y_test_pred.round(1)
})
print(test_results.sort_values('Hours'))
print()
```

```
Test Set - Actual vs Predicted:
    Hours  Actual Score  Predicted Score
0       1            50             55.0
9      10            94             94.4
10     11            96             98.8
```

```python
# Step 7: Make a prediction for a new student
new_student_hours = 7.5
predicted_score = model.predict([[new_student_hours]])[0]

print(f"New student studied {new_student_hours} hours → Predicted score: {predicted_score:.1f}")
print()
```

```
New student studied 7.5 hours → Predicted score: 83.5

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have v
  warnings.warn(
```

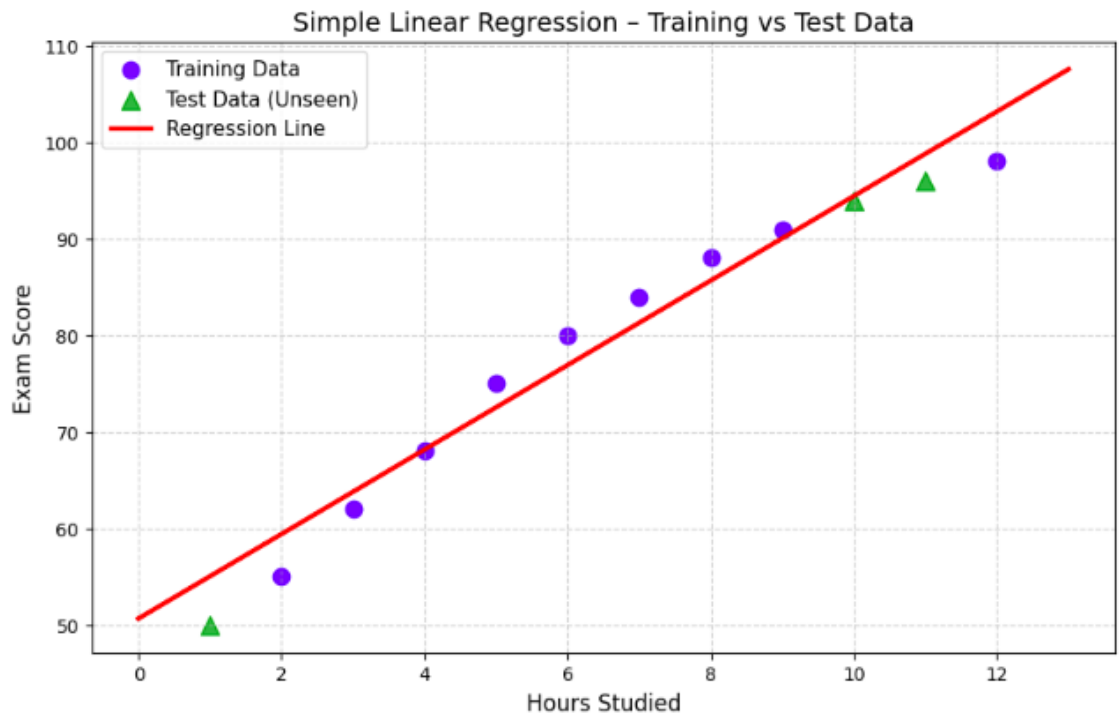
```python
# Step 8: Visualize everything
plt.figure(figsize=(10, 6))

# Plot training data
plt.scatter(X_train, y_train, color='blue', label='Training Data', alpha=0.7, s=80)

# Plot test data
plt.scatter(X_test, y_test, color='green', label='Test Data (Unseen)', alpha=0.7, s=100, marker='^')

# Plot the regression line
X_line = pd.DataFrame({'Hours_Studied': range(0, 14)})
y_line = model.predict(X_line)
plt.plot(X_line, y_line, color='red', linewidth=2.5, label='Regression Line')

plt.xlabel('Hours Studied', fontsize=12)
plt.ylabel('Exam Score', fontsize=12)
plt.title('Simple Linear Regression - Training vs Test Data', fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```




## Multiple Linear Regression

- When there are **multiple predictors**, the equation is simply extended to accommodate them:

$$Y = b_0 + b_1X_1 + b_2X_2 + ... + b_pX_p + e$$

- Multiple Linear Regression (MLR) is a statistical technique used to model the relationship between one dependent variable (Y) and two or more independent variables ($X_1$, $X_2$, ..., $X_n$)
- Instead of a line, we now have a linear model—the relationship between each coefficient and its variable (feature) is linear.

- All of the other concepts in simple linear regression, such as **fitting by least squares** and the definition of **fitted values** and **residuals**, **extend to the multiple linear regression** setting. For example, the fitted values are given by:

$$\hat{Y}_i = \hat{b}_0 + \hat{b}_1 X_{1,i} + \hat{b}_2 X_{2,i} + \ldots + \hat{b}_p X_{p,i}$$

**Multiple Linear regression**

```python
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

```python
# Step 1: Create sample dataset
# Predicting House Price (in $1000s) based on:
# - Size (sq ft)
# - Number of Bedrooms
# - Number of Bathrooms

data = {
    'Size_sqft':    [1000, 1200, 1500, 1800, 2000, 2200, 2500, 2800, 3000, 3200],
    'Bedrooms':     [2,    2,    3,    3,    4,    4,    4,    5,    5,    5],
    'Bathrooms':    [1,    1.5,  2,    2,    2.5,  3,    3,    3.5,  4,    4],
    'Price_1000s':  [200,  240,  300,  350,  400,  450,  520,  580,  620,  680
}

df = pd.DataFrame(data)
print("House Price Dataset:")
print(df)
print()
```

```
House Price Dataset:
   Size_sqft  Bedrooms  Bathrooms  Price_1000s
0       1000         2        1.0          200
1       1200         2        1.5          240
2       1500         3        2.0          300
3       1800         3        2.0          350
4       2000         4        2.5          400
5       2200         4        3.0          450
6       2500         4        3.0          520
7       2800         5        3.5          580
8       3000         5        4.0          620
9       3200         5        4.0          680
```

```python
# Step 2: Prepare features (X) and target (y)
X = df[['Size_sqft', 'Bedrooms', 'Bathrooms']]   # Multiple features
y = df['Price_1000s']                            # Target variable
```

```python
# Step 3: Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,          # 30% for testing (small dataset)
    random_state=42         # reproducible split
)

print(f"Training samples: {len(X_train)}")
print(f"Testing samples:  {len(X_test)}")
print()
```

```
Training samples: 7
Testing samples:  3
```

```python
# Step 4: Create and train the Multiple Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

```python
# Step 5: Show the learned regression equation
intercept = model.intercept_
coefficients = model.coef_

print("Multiple Linear Regression Equation:")
print(f"Price ($1000s) = {intercept:.2f}")
print(f"          + {coefficients[0]:.2f} × Size_sqft")
print(f"          + {coefficients[1]:.2f} × Bedrooms")
print(f"          + {coefficients[2]:.2f} × Bathrooms")
print()
```

```
Multiple Linear Regression Equation:
Price ($1000s) = -9.50
          + 0.20 × Size_sqft
          + -18.75 × Bedrooms
          + 33.13 × Bathrooms
```

```python
# Step 6: Make predictions on the test set
y_test_pred = model.predict(X_test)
# Calculate metrics on the TEST set
mae   = mean_absolute_error(y_test, y_test_pred)
mse   = mean_squared_error(y_test, y_test_pred)
rmse  = np.sqrt(mse)
r2    = r2_score(y_test, y_test_pred)

print("\n" + "="*60)
print("PERFORMANCE METRICS - MULTIPLE LINEAR REGRESSION")
print("="*60)
print(f"MAE   (Mean Absolute Error)     : {mae:>10.2f} ($1000s)")
print(f"MSE   (Mean Squared Error)      : {mse:>10.2f}")
print(f"RMSE  (Root Mean Squared Error) : {rmse:>10.2f} ($1000s)")
print(f"R²    Score                      : {r2:>10.4f}  ({r2*100:>5.1f}% variance explained)")
print("="*60)
print()
```

```

============================================================
PERFORMANCE METRICS - MULTIPLE LINEAR REGRESSION
============================================================
MAE   (Mean Absolute Error)     :       9.60 ($1000s)
MSE   (Mean Squared Error)      :     108.88
RMSE  (Root Mean Squared Error) :      10.43 ($1000s)
R²    Score                      :     0.9955  ( 99.5% variance explained)
============================================================
```

```python
# Step 6: Make predictions on the test set
y_test_pred = model.predict(X_test)

# Show actual vs predicted for test houses
print("Test Set - Actual vs Predicted Prices:")
test_results = pd.DataFrame({
    'Size_sqft': X_test['Size_sqft'],
    'Bedrooms':  X_test['Bedrooms'],
    'Bathrooms': X_test['Bathrooms'],
    'Actual_Price': y_test,
    'Predicted_Price': y_test_pred.round(1)
})
print(test_results.sort_values('Size_sqft'))
print()
```

```
Test Set - Actual vs Predicted Prices:
   Size_sqft  Bedrooms  Bathrooms  Actual_Price  Predicted_Price
1       1200         2        1.5           240            244.9
5       2200         4        3.0           450            459.0
8       3000         5        4.0           620            634.9
```

```python
# Step 7: Predict price for a new house
new_house = pd.DataFrame({
    'Size_sqft': [2400],
    'Bedrooms':  [4],
    'Bathrooms': [3]
})

predicted_price = model.predict(new_house)[0]
print(f"New house: 2400 sqft, 4 bedrooms, 3 bathrooms")
print(f"→ Predicted price: ${predicted_price:.1f}k")
print()
```

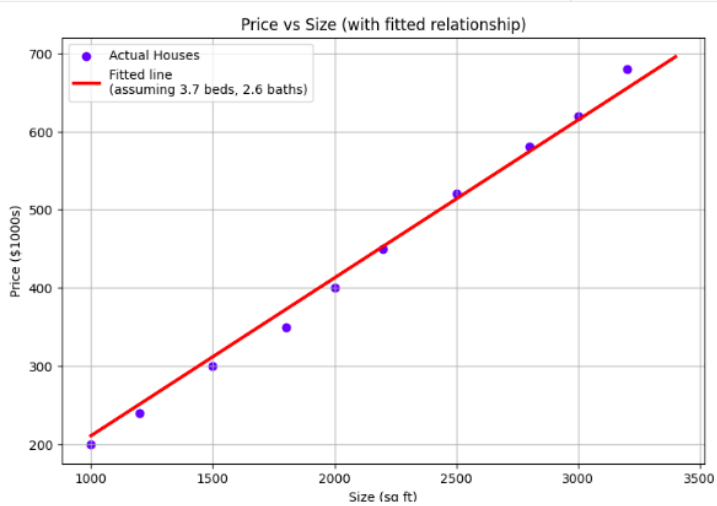
```
New house: 2400 sqft, 4 bedrooms, 3 bathrooms
→ Predicted price: $499.4k
```

```python
# Create range of sizes, keep other features at mean value
mean_bed = df['Bedrooms'].mean()
mean_bath = df['Bathrooms'].mean()

size_range = pd.DataFrame({
    'Size_sqft': range(1000, 3500, 200),
    'Bedrooms':  [mean_bed] * 13,
    'Bathrooms': [mean_bath] * 13
})

predicted_vs_size = model.predict(size_range)

plt.figure(figsize=(9, 6))
plt.scatter(df['Size_sqft'], df['Price_1000s'], color='blue', label='Actual Houses')
plt.plot(size_range['Size_sqft'], predicted_vs_size, color='red', linewidth=2.5,
         label=f'Fitted line\n(assuming {mean_bed:.1f} beds, {mean_bath:.1f} baths)')
plt.xlabel('Size (sq ft)')
plt.ylabel('Price ($1000s)')
plt.title('Price vs Size (with fitted relationship)')
plt.legend()
plt.grid(True)
plt.show()
```



## Lab Exercise

### Task 1: Simple Linear Regression (Predicting House Prices)

**Scenario:** A real estate agency wants to predict house prices based solely on the square footage of the living area.

Square Footage (X) vs House Price in USD

(Y) data1 = {

'SqFt': [1500, 1800, 2400, 3000, 3500, 4000, 4500, 5000],

'Price': [300000, 350000, 450000, 500000, 580000, 650000, 720000, 800000]

}

1. **Split:** Use a 75/25 split for training and testing.
2. **Model:** Train a LinearRegression model.
3. **Predict:** Predict the price for a house with **2,800 SqFt**.
4. **Evaluate:** Calculate and print the **MAE**, **RMSE**, and **R2 Score**.
5. **Visualize:** Plot the actual data points and the regression line.

## Task 2: Multiple Linear Regression (Startup Success)

**Scenario:** An investment firm wants to predict the **Profit** of a startup based on its spending in different departments: R&D, Administration, and Marketing.

# Spending in USD (X1, X2, X3) vs Profit (Y)

data2 = {

'RD_Spend': [165349, 162597, 153441, 144372, 142107, 131876, 134615],

'Admin_Spend': [136897, 151377, 101145, 118671, 91391, 99814, 147198],

'Marketing_Spend': [471784, 443898, 407934, 383199, 366168, 362861, 127716],

'Profit': [192261, 191792, 191050, 182901, 166187, 156991, 156122]

}

1. **Setup:** Define X (3 columns) and Y (Profit).
2. **Training:** Split your data (80% Training, 20% Testing) and train a **Multiple Linear Regression** model.

3. **Investigation:** Print the **Coefficients** (β) for all three spending categories.
4. **Analysis:** Which department has the highest positive impact on Profit based on the coefficients?
5. **Scoring:** Provide the **R2 Score** and explain in one sentence what it tells you about this specific model

**Task 3: Student Performance Analytics**

LINK TO DATASET

**Goal:** Build a Machine Learning model to predict a student's **Performance Index** based on their study habits and lifestyle.

1. **Setup:** Define X and Y.
2. **Split your data** (80% Training, 20% Testing) and train a **Multiple Linear Regression** model.
3. **Training:** Train the Multiple Linear Regression model.
4. **Investigation:** Print the **Coefficients** (β) for all three spending categories.
5. **Evaluation:** Print the **MAE**, **RMSE**, and **R2 Score** for your model.

After running your code, write a 2-sentence interpretation for each:

- **R2 Score:** Does your model explain most of the variation in grades?
- **MAE vs. RMSE:** Is your RMSE much higher than your MAE? If so, why does that mean?
- **Actionable Advice:** Based on the **Coefficients**, should a student focus more on "Sleep" or "Sample Papers" to improve their score?

**Question :** If your MAE is 5,000 and your RMSE is 12,000, what does that tell you about your model's errors?

**Question :** If the R2 score is 0.45, is this model reliable for making business decisions? Why or why not?